Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 6_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Anitha is working on packing expensive artifacts into a container that has a limited weight capacity. Each artifact has a weight and a value, and Anitha wants to maximize the total worth of artifacts she can carry, even if it means taking only a fraction of an artifact when the container is nearly full.

The maximum worth is calculated as:

Maximum Worth = Σ (value[i] * fraction_taken[i]),

where fraction_taken[i] = min(1, remaining_capacity / weight[i])

Write a program to ensure that Anitha can calculate the maximum worth of objects she can carry in the container.

Input Format

The first line contains an integer N, representing the number of items.

The second line contains N space-separated integers representing the weights of the items.

The third line contains N space-separated integers representing the values of the items.

The fourth line contains an integer representing the capacity of the container.

Output Format

The output prints a single line showing the total maximum worth of objects carried as a double with exactly two decimal places, in the following format:

"Objects worth Rs.X.YY"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
10 20 30
60 100 120
50
Output: Objects worth Rs.240.00

Answer

```
#include <stdio.h>

struct Item {
    int weight;
    int value;
    double ratio;
};
```

```
void sortItems(struct Item items[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (items[j].ratio < items[j + 1].ratio) {
                struct Item temp = items[j];
                items[j] = items[j + 1];
                items[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n, capacity;
    struct Item items[10];

    if (scanf("%d", &n) != 1) return 0;

    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i].weight);
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &items[i].value);
        items[i].ratio = (double)items[i].value / items[i].weight;
    }

    scanf("%d", &capacity);

    sortItems(items, n);

    double totalWorth = 0.0;
    int currentWeight = 0;

    for (int i = 0; i < n; i++) {
        if (currentWeight + items[i].weight <= capacity) {
            currentWeight += items[i].weight;
            totalWorth += items[i].value;
        } else {
            int remain = capacity - currentWeight;
            totalWorth += items[i].value * ((double)remain / items[i].weight);
            break;
        }
```

```
    }
    printf("Objects worth Rs.%.2f\n", totalWorth);
    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Raju is a professional hiker preparing for a long trekking expedition. He has a backpack that can hold a maximum weight of W kilograms. There are N essential items you want to carry, each with a specific weight and importance value (utility score). His goal is to maximize the total importance value of the items you take while ensuring that the total weight does not exceed the backpack's capacity.

He can either include or exclude each item—meaning he cannot take fractions of an item (e.g., you cannot take half of a tent). Given the weights and importance values of the items, determine the maximum possible importance value he can carry in his backpack. Help him to achieve his task.

Input Format

The first line contains two integers, N (number of items) and W (maximum weight the backpack can carry).

The second line contains N integers, representing the weights of the items.

The third line contains N integers, representing the importance values of the items.

Output Format

The output prints a single integer, the maximum importance value that can be achieved without exceeding the weight limit.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 4 5
2 3 4 5
3 4 5 6

Output: 7

Answer

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n, W;
    if (scanf("%d %d", &n, &W) != 2) return 0;

    int wt[n], val[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &wt[i]);
    }
    for (int i = 0; i < n; i++) {
        scanf("%d", &val[i]);
    }

    // Using a 1D DP array to optimize space since W can be up to 10^5
    // dp[w] will store the maximum importance value for capacity w
    int dp[W + 1];

    for (int i = 0; i <= W; i++) {
        dp[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        // We traverse backwards to ensure each item is used only once
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = max(dp[w], val[i] + dp[w - wt[i]]);
```

```
        }
    }

    printf("%d\n", dp[W]);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Rohit is responsible for scheduling flights for an airline company. He has a list of N flights numbered from 0 to N-1. Some flights have specific constraints that require them to depart after other flights.

Write a program to help Rohit determine if it's possible to create a flight schedule that adheres to these constraints using Warshall's algorithm.

Example 1

Input:

5

0 1

1 2

2 3

3 4

4 5

Output:

Yes

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

There are no circular dependencies or conflicts in the schedule, so it is possible to create a flight schedule that adheres to the constraints.

Example 2

Input:

6

0 1

1 2

2 3

3 4

4 5

5 0

Output:

No

Explanation:

Flight 0 must depart after Flight 1.

Flight 1 must depart after Flight 2.

Flight 2 must depart after Flight 3.

Flight 3 must depart after Flight 4.

Flight 4 must depart after Flight 5.

Flight 5 must depart after Flight 0.

There is a circular dependency in the flight constraints. Therefore, it is not

possible to create a flight schedule that follows the constraints.

Input Format

The first line of input consists of an integer N, representing the number of flights.

The following N lines consist of two space-separated integers each: the flight number and the flight number that must depart before it.

Output Format

If it is possible to create a flight schedule that adheres to the constraints, print "Yes".

Otherwise, print "No".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
0 1
1 2
2 3
3 4
4 5
Output: Yes

Answer

```
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int matrix[20][20] = {0};
    int max_node = 0;

    for (int i = 0; i < n; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
```

```
        matrix[u][v] = 1;
        if (u > max_node) max_node = u;
        if (v > max_node) max_node = v;
    }

    int size = max_node + 1;

    for (int k = 0; k < size; k++) {
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                if (matrix[i][k] && matrix[k][j]) {
                    matrix[i][j] = 1;
                }
            }
        }
    }

    int hasCycle = 0;
    for (int i = 0; i < size; i++) {
        if (matrix[i][i] == 1) {
            hasCycle = 1;
            break;
        }
    }

    if (hasCycle) {
        printf("No\n");
    } else {
        printf("Yes\n");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

A traveler needs to find the shortest path between every pair of cities in a road network. The cities are connected by roads, each with a specific travel

cost. To do this, the traveler uses the Floyd-Warshall Algorithm to calculate the shortest distances between every pair of cities in a given edge-weighted undirected graph.

Example1

Input:

4

3

0 1 2

1 2 3

2 3 4

Output:

Original matrix

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Shortest path matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Explanation:

Given: 4 nodes (0, 1, 2, 3) and 3 edges: 0 1 (weight 2), 1 2 (weight 3), 2 3 (weight 4)

Any node pair without a direct edge is considered INF (unreachable initially).

Step 1: Construct Initial Graph (Original Matrix)

0 2 INF INF

2 0 3 INF

INF 3 0 4

INF INF 4 0

Diagonal elements (self to self) are 0.If no direct path exists, it's INF.Step 2: Running Floyd-Warshall Algorithm

Using node 0 as an intermediate: No updates since it only connects to node 1.

Using node 1 as an intermediate: 0 2 via 1: 0 1 (2) + 1 2 (3) = 5

2 0 also updates to 5.

Using node 2 as an intermediate: 0 3 via 2: 0 2 (5) + 2 3 (4) = 9

1 3 via 2: 1 2 (3) + 2 3 (4) = 7

Using node 3 as an intermediate: No further updates.

Step 3: Final Shortest Path Matrix

0 2 5 9

2 0 3 7

5 3 0 4

9 7 4 0

Input Format

The first line contains an integer V representing the number of cities (vertices) in the network.

The second line contains an integer E representing the number of roads (edges) in the network.

The next E lines each contain three space-separated integers u, v, w, where u and v represent the cities connected by a road, and w represents the travel cost (weight) of the road.

Output Format

The first line of output prints the original matrix, where each entry represents the direct travel cost between two cities. If there is no direct road between two cities, represent the cost as INF.

Then, print the shortest path matrix, where each entry represents the shortest travel cost between every pair of cities, computed using the Floyd-Warshall algorithm.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
3
0 1 2
1 2 3
2 3 4
Output: Original matrix
0 2 INF INF
2 0 3 INF
INF 3 0 4
INF INF 4 0

Shortest path matrix
0 2 5 9
2 0 3 7
5 3 0 4
9 7 4 0

Answer

```
#include <stdio.h>

#define INF 1e9

void printMatrix(int V, long long matrix[V][V]) {
   for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
         if (matrix[i][j] == INF) {
            printf("INF ");
         } else {
```

```
            printf("%lld ", matrix[i][j]);
        }
    }
    printf("\n");
  }
}

int main() {
  int V, E;
  if (scanf("%d %d", &V, &E) != 2) return 0;

  long long matrix[V][V];

  for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
      if (i == j) {
        matrix[i][j] = 0;
      } else {
        matrix[i][j] = INF;
      }
    }
  }

  for (int i = 0; i < E; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);
    if (u < V && v < V) {
      matrix[u][v] = w;
      matrix[v][u] = w;
    }
  }

  printf("Original matrix\n");
  printMatrix(V, matrix);

  for (int k = 0; k < V; k++) {
    for (int i = 0; i < V; i++) {
      for (int j = 0; j < V; j++) {
        if (matrix[i][k] != INF && matrix[k][j] != INF) {
          if (matrix[i][k] + matrix[k][j] < matrix[i][j]) {
            matrix[i][j] = matrix[i][k] + matrix[k][j];
          }
```

```
            }
        }
    }
}

    printf("\nShortest path matrix\n");
    printMatrix(V, matrix);

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

A person starts walking from position X = 0, find the probability of reaching exactly on X = N if she can only take either 2 steps or 3 steps. The probability for step length 2 is given i.e. P, and the probability for step length 3 is 1 – P.

Write a program for the same.

Note: Use Dynamic Programming

Example

Input: N = 5, P = 0.20

Output: 0.32

Explanation:

There are two ways to reach 5.

2+3 with probability = 0.2 * 0.8 = 0.16

3+2 with probability = 0.8 * 0.2 = 0.16

So, total probability = 0.32.

Input Format

The input consists of an integer N representing the target position and a double-point number P representing the probability of taking a step of length 2, separated by a space.

Output Format

The output prints a double-point number representing the probability of reaching the exact position N, formatted to two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 0.20
Output: 0.32

Answer

```
#include <stdio.h>

int main() {
    int n;
    double p;

    if (scanf("%d %lf", &n, &p) != 2) return 0;

    double q = 1.0 - p;
    double dp[21];

    for (int i = 0; i <= n; i++) {
        dp[i] = 0.0;
    }

    dp[0] = 1.0;

    for (int i = 2; i <= n; i++) {
        double prob2 = 0.0;
        double prob3 = 0.0;

        if (i - 2 >= 0) {
            prob2 = dp[i - 2] * p;
        }
```

```
        if (i - 3 >= 0) {
            prob3 = dp[i - 3] * q;
        }

        dp[i] = prob2 + prob3;
    }

    printf("%.2f\n", dp[n]);

    return 0;
}
```

Status : Correct **Marks : 10/10**